



Building Semantic Models in Databricks with Unity Catalog Metric Views

Databricks Data + AI Summit
2026

Databricks Academy



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).

Meet Your Teaching Team

- We have a team ready to support you in today's class
 - **Instructor:** Koji Kawamura
 - **TA:** Satish Garla
- Please raise your hand at any point throughout the class and a TA will make their way to you to assist you!



Goals for Our Session Today

- Explain the role and benefits of semantic modeling with metric views in Databricks
- Define and manage metric views
- Model advanced business metrics, including windowed measures
- Integrate metric views into Databricks dashboards, Genie, and SQL workflows
- Be aware of best practices for security, governance, and maintenance of metric views



Agenda

Building Semantic Models in Databricks with Unity Catalog Metric Views	Time
Semantic Layer Fundamentals	25 mins
Defining and Consuming Metric Views	45 mins
Designing and Composing Metric Views	60 mins
Metric Views in Production	20 mins
Summary and Next Steps	60 mins



Lab Exercise Environment

Technical Details



- Your lab environment is provided by Vocareum – it will open in a new tab
- It has been configured with the permissions and resources required to accomplish the tasks outlined in this course
- Third party cookies must be enabled in your browser for Vocareum's user experience to work properly.
- Make sure to enable pop ups!



Before we get started...

A few quick notes

- Be aware, Databricks is a highly innovative company and through our courses we try to give you the latest and greatest.
- With all of the exciting changes being announced at Summit, you might find inconsistencies with product naming – no worries – we'll guide you through the latest today!
- Reach out to your Instructor or TA for help, or leverage Genie Code for instant assistance. Read more about Genie Code [here](#).

Enjoy the course!

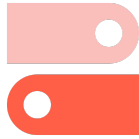


Course Prerequisites (REQUIRED)



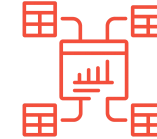
Fundamental Knowledge of the Databricks Platform

- Databricks Workspaces
- Creating Notebooks or SQL Queries
- Delta Lake and the Medallion architecture
- Unity Catalog data objects (catalogs, schemas, tables, etc.)
- Basic compute concepts (SQL Warehouses vs Clusters)



Intermediate Level Databricks SQL (DBSQL)

- SELECT statements
- Filtering data
- SQL functions
- Joins



Understanding of Data Modeling Concepts

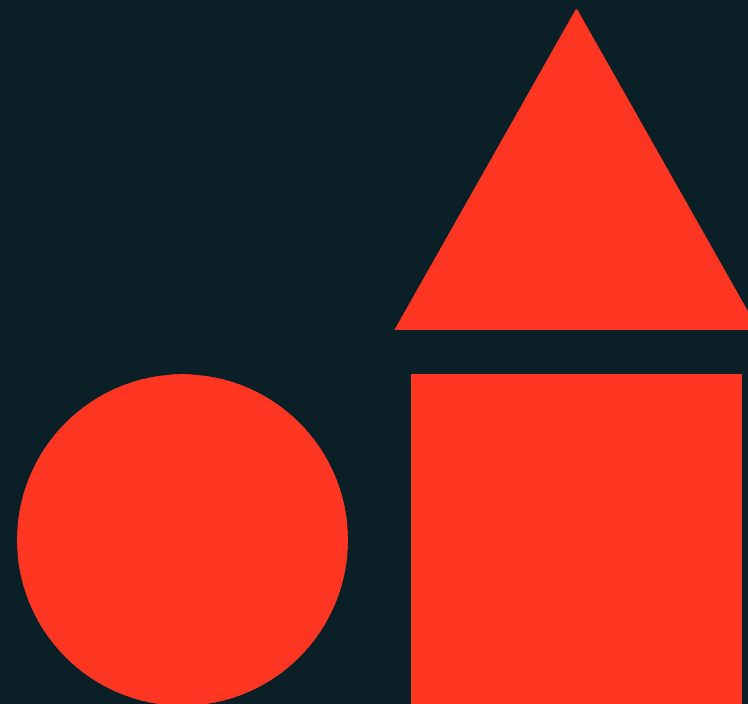
- Understanding of snowflake and star schemas





Semantic Layer Fundamentals

Building Semantic Models in Databricks with UC Metric Views



Section Learning Objectives

- Explain the purpose and value of a semantic layer
- Explain how Unity Catalog acts as a semantic layer
- Explain the use case and benefits of metric views
- Identify differences between metric views and traditional SQL views
- Explain how metric views fit in the Databricks ecosystem



Agenda

Semantic Layer Fundamentals	Lecture	Demo	Lab
How metric views empower users	✓		
How metric views fit into Databricks	✓		
Design patterns for metric views	✓		





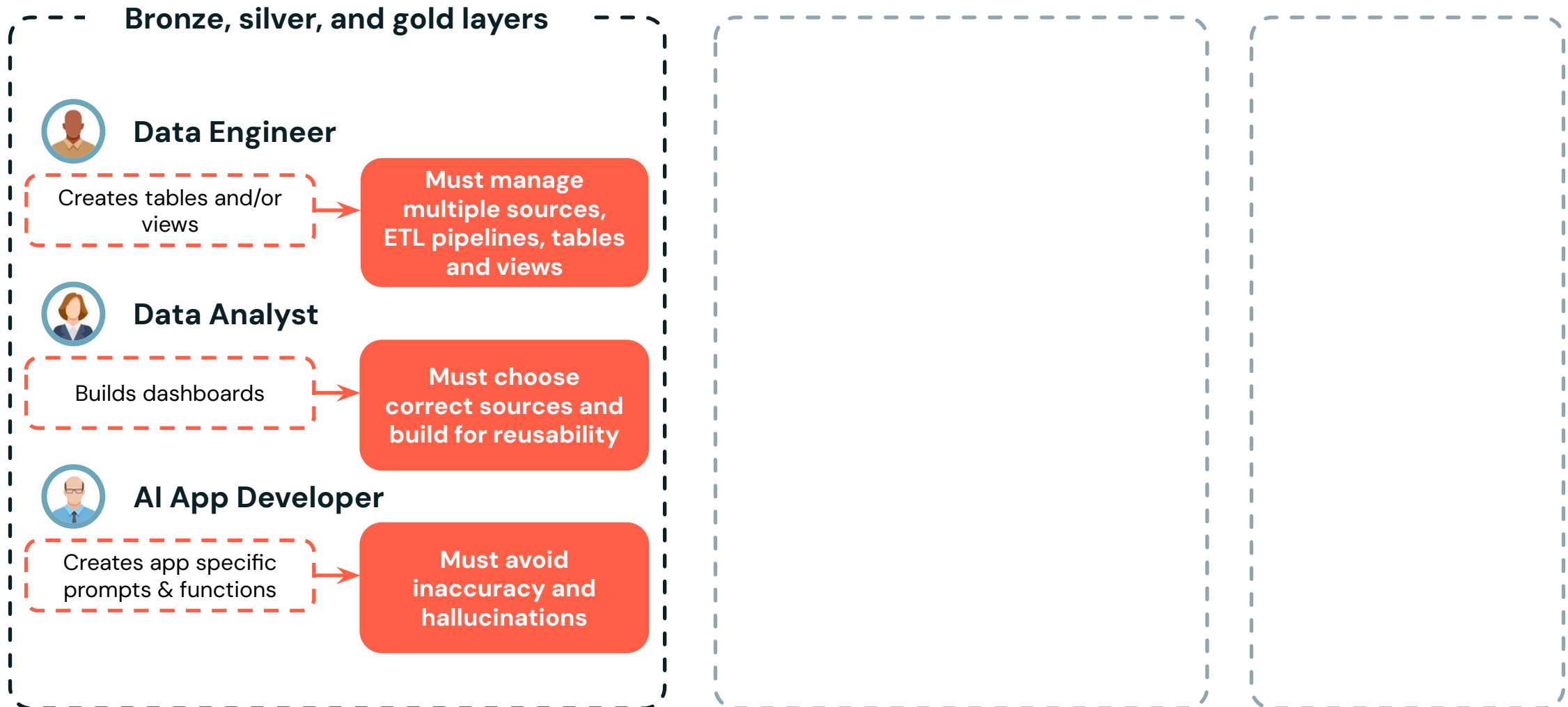
Semantic Layer Fundamentals

LECTURE

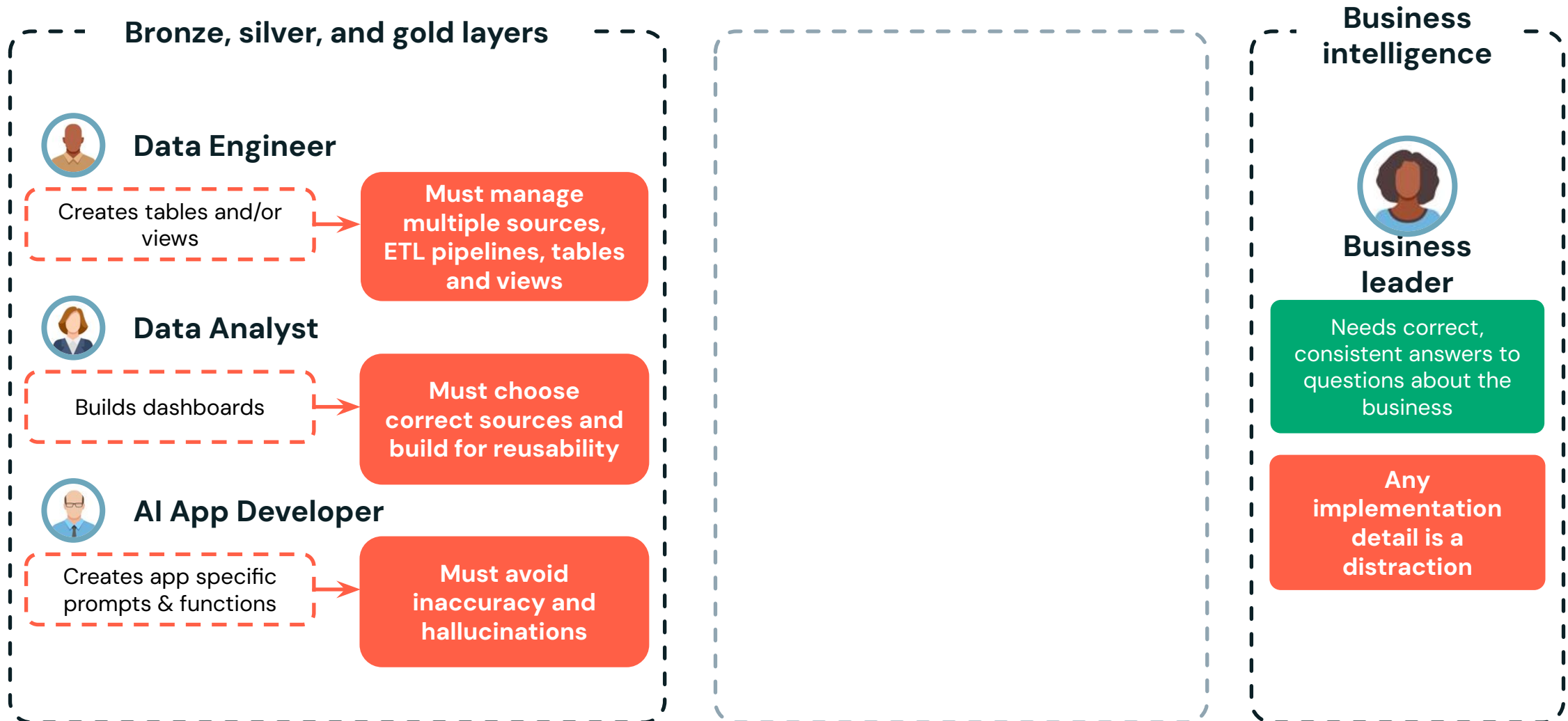
How metric views empower users



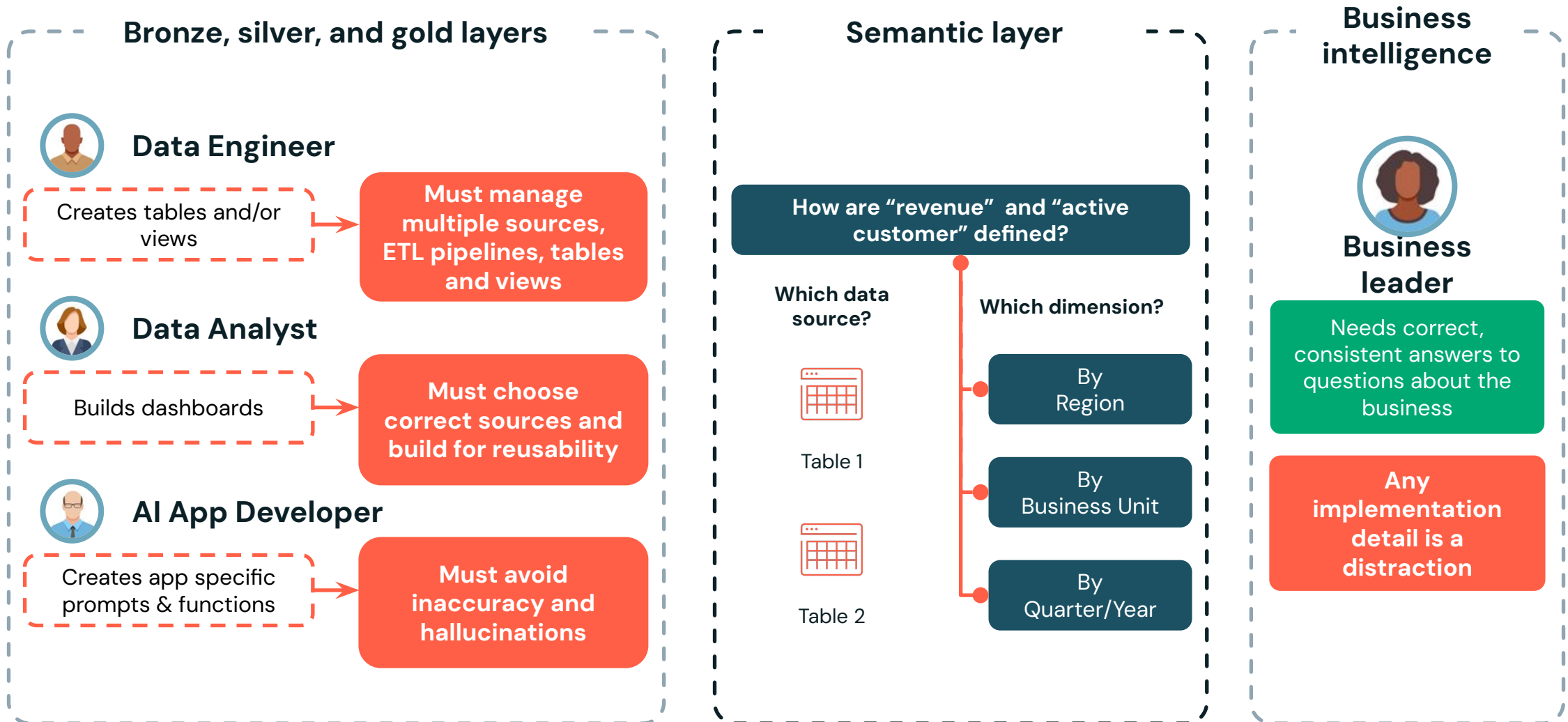
Your data + AI team's sphere of concerns



Your data consumers' sphere of concerns

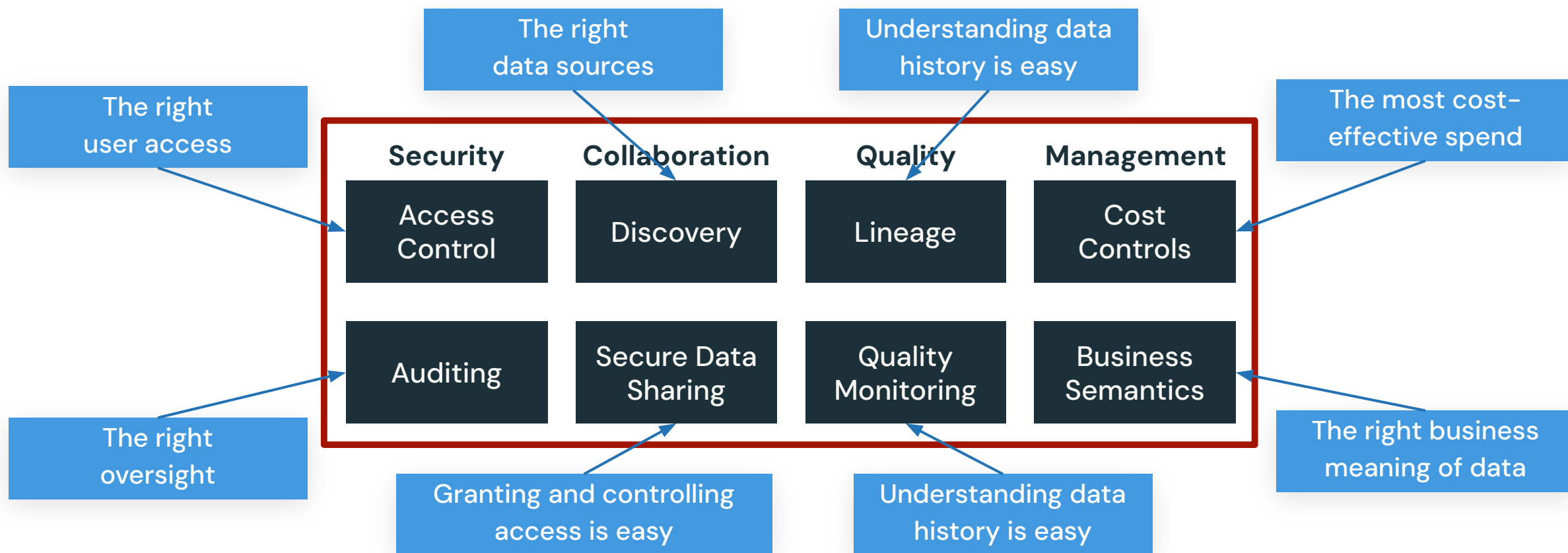


How to bridge the gap: the semantic layer



How to build a semantic layer: Unity Catalog

Databricks's solution for unified open governance for all data + AI



How to build a semantic layer: Unity Catalog

Security	Collaboration	Quality	Management
Access Control	Discovery	Lineage	Cost Controls
Auditing	Secure Data Sharing	Quality Monitoring	Business Semantics

- Metadata on each object gives business context
- Tags capture data relationships across the business
- Metric views define single sources of truth for key performance indicators



Metric views abstract, unify, and enforce

- A smooth surface for **business data consumers**, uncluttered by engineering concerns
- An audit-friendly environment for a single definition of all KPIs
- A data product, built and maintained by the data + AI team, for the business
- *A semantic model of the data, customized for what business decision makers care most about*





Module Name

LECTURE

How metric views fit into Databricks



Metric views vs traditional views



Traditional SQL view

- Defined and stored as **SQL**
- Basis for saved **queries, dashboards**, and **Genie spaces**
- Suited for general data engineering or data analysis scenarios



Unity Catalog Metric View

- Defined using a **YAML markup language** on top of SQL
- *Defined once, reused everywhere*
 - Ensures **consistent metric definitions** across tools & teams
 - The **ideal** basis for saved queries, dashboards, and Genie spaces
 - Enables **simpler queries** for consumers

Both are **virtual tables**

Both are **fully governed** using **Unity Catalog permissions**

Both can be used in **queries** and **Lakeflow Declarative Pipelines**



What about materialized views?

Materialized Views are Unity Catalog managed tables that **physically store the results of a query**, delivering faster performance on frequently used data.

Why materialize a traditional **SQL view**?

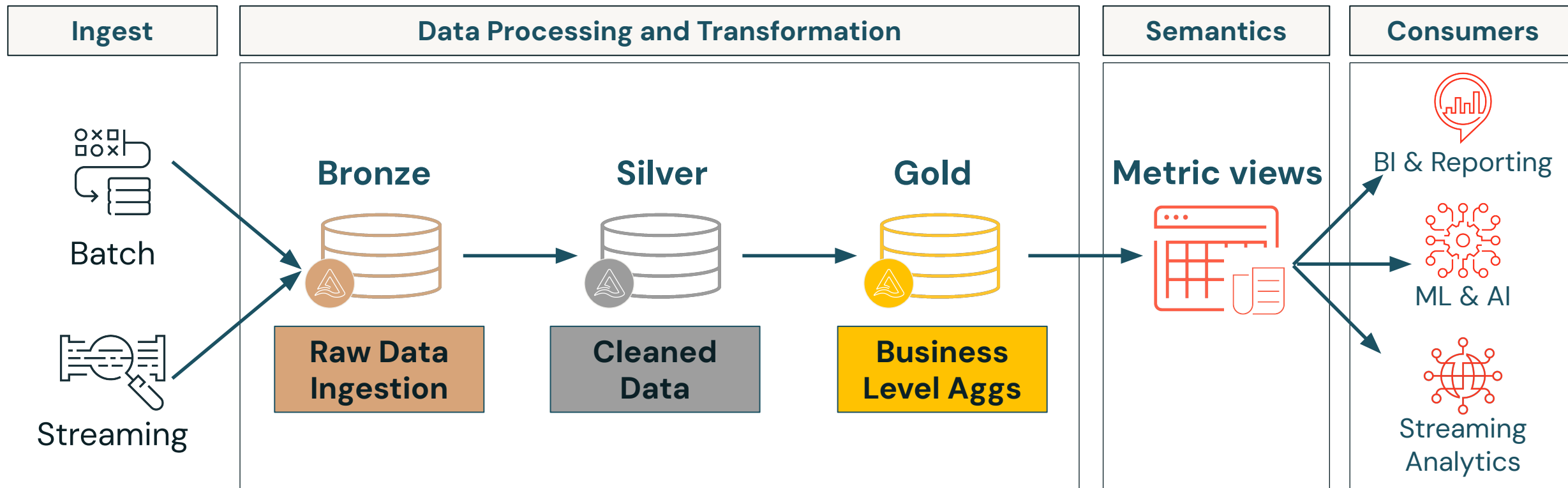
- Ensure optimized response to often-performed queries
- Establish a tradeoff among cost, cost predictability, response time, and data freshness

Why materialize a Unity Catalog **metric view**?

- Same reasons!
- Metric views, by design, capture the queries desired most by the business
- ***Coming to public preview soon***



Where do metric views fit in the Databricks architecture?





Module Name

LECTURE

Design patterns for metric views



Rules for designing measures and dimensions



When you are designing *measures*

- Offer only measures **relevant** to the business
 - Include only KPIs your data consumers express a need for
 - Don't include measures just because they are easy or cool
- **Abstract** away all implementation details such as table or field names
- **You can't** use GROUP BY clauses in measure definitions



When you are designing *dimensions*

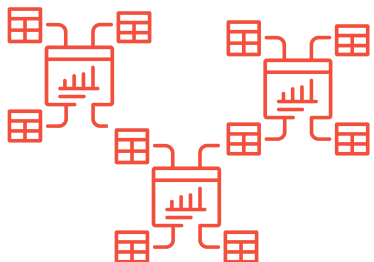
- Offer only those ways of slicing measures that are **relevant** to the business
 - Include only dimensions your data consumers express a need for
 - Don't include dimensions just because they are easy or cool
- **Abstract** away all implementation details such as table or field names
- **You can't** use SQL aggregations in dimension definitions



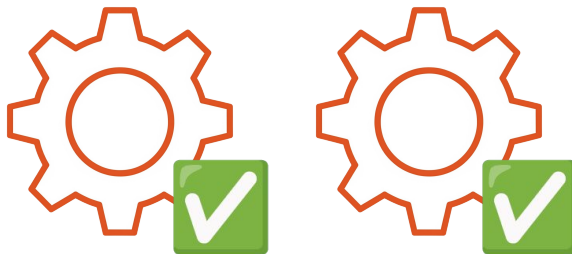
Metric views help you achieve these design goals



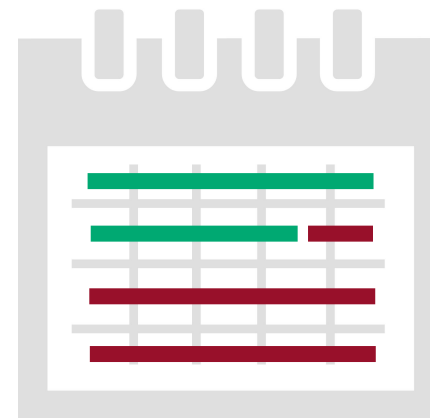
*Abstract away
data-engineering
concerns*



*Define business logic
once, use it many times*



*Hide changes to
underlying data schemas*



Design pattern: hide complexity from consumers

Problem

- Business data scattered across many tables
- Opaque field names
- Complex joins required
- High engineering effort needed to build correct dashboards

Action

- Build a metric view that points to data where it is
- Capture and document the complex joins in the metric view
- Capture and document opaque field names
- Expose measures and dimensions with simple names easy to understand by the business

Result

- Business needs no knowledge of underlying field names, schema layouts, or joins
- Dashboard construction is greatly simplified



Design pattern: unify multiple KPI definitions

Problem

- Dashboards based on different SQL queries with no governance
- Aggregations defined differently in different contexts
- Result: inconsistent views of the same data

Action

- Get all parties to agree on how measures should be defined
- Capture these agreements in a metric view
- Change dashboards and Genie spaces to query the metric view instead of underlying tables

Result

- Dashboards are consistent
- Definitions of KPIs are now maintained in a reviewable document



Design pattern: manage schema changes

Problem

- On Saturday night, we're going to restructure a critical table on which many dashboards depend
- But the business will suffer if those dashboards stop working

Action

- Before the changeover, refactor the dashboards to use a metric view instead of the underlying tables
- Stage and review a new version of the metric view that exposes the same measures and dimensions with new definitions
- Push live the revised metric view with the table changes

Result

- Data consumers are insulated from the data structure change



Design pattern: facilitate team ownership

Problem

- Lines of demarcation are fuzzy for critical business metrics
- Ownership of definitions unclear

Action

- Create metric views that follow ownership boundaries
 - Only the owning team can modify the definition
 - Other team can use the view
- Dimensions and measures owned by one team can be used in views owned by another team

Result

- Ownership is clear and documented



Design pattern: guard against erroneous reaggregation

Problem

- Data consumers are incorrectly...
 - Summing daily account balances
 - GROUPing by hour on data already smoothed to 24-hour days
 - Spending resources on expensive, unsupported views

Action

- Create a metric view that only allows approved, meaningful dimensions
- Replace data consumers' access to tables with access to the metric view

Result

- Erroneous or wasteful aggregations or joins are blocked



Design pattern: optimize for broadcast joins

Problem

- Data consumers are not systematically choosing small lookup tables for left joins
 - Result: slow query performance or excess resource consumption

Action

- Create a metric view that is pre-joined with small lookup tables
- Replace data consumers' access to underlying tables with access to the metric view

Result

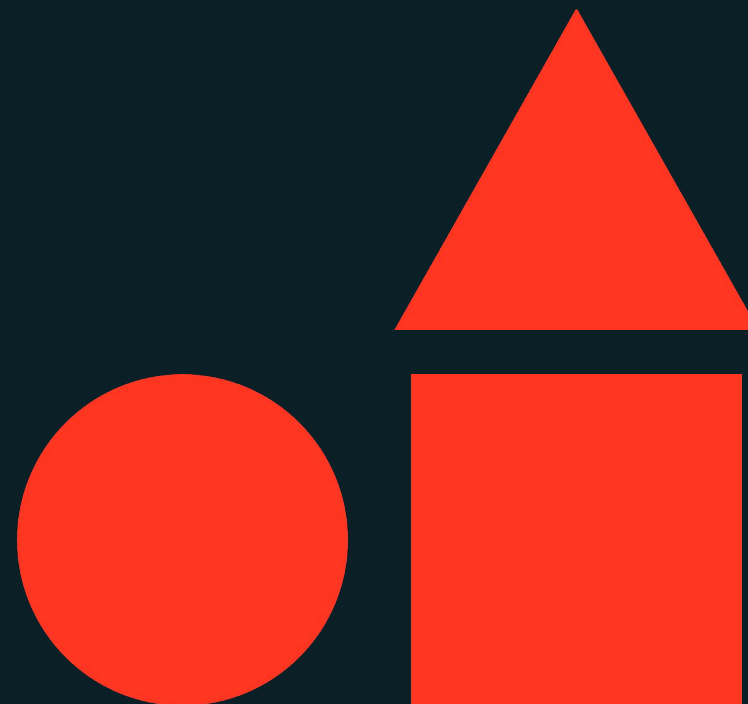
- Because these tables fit into the memory of Spark worker nodes, they can be broadcast
- Efficient joins are now performed automatically





Defining and Consuming Metric Views

Building Semantic Models in Databricks with UC Metric Views



Section Learning Objectives

- Understand the structure of metric views
- Build semantically rich metric views



Agenda

Defining and Consuming Metric Views	Lecture	Demo	Lab
Model and define metric views	✓		
Create metric views	✓		
Create a simple metric view		✓	
Query and consume metric views	✓		
Consume a Simple Metric View		✓	
Create and Consume a Simple Metric View			✓





Defining and Consuming Metric Views

LECTURE

Model and define metric views



Metric views are defined as YAML text files

The YAML format is human-readable and suitable for version control



Why YAML?

YAML lets you define data structures with sophisticated layouts

Because it is a text file:

- It can **contain SQL expressions**
- It can be **checked into git** or other versioning systems
- It can be **shared for review** and approval as simply as a text document

A sample YAML file for a metric view

```
version: 1.1
source: ...
joins:
  - name: ...
    source: ...
    on: ...
dimensions:
  - name: ...
    expr: ...
measures:
  - name: ...
    expr: ...
```



Metric View Key Components

Each metric view defines **measures** and **dimensions**



What is a Measure?

- Numeric values the business depends on (sales, revenue, clicks). Typically, **critical KPIs**.
- Defined using SQL expressions on source data.
 - Can be simple fields or calculations using one or more source fields
 - Typically including **aggregations** such as SUM, AVG, COUNT, MIN, MAX

Example: `total_sales` , defined as:

```
SUM(source.ordertotal)
```



What is a Dimension?

- Attributes or categories that the business needs those measures to be grouped by (date range, region, product).
- Defined using SQL expressions on source data.
 - Can be simple fields or calculations using one or more source fields,
 - **No aggregations**

Example: `fiscal_quarter` , defined as:

```
QUARTER(source.orderdate)
```



Indent YAML consistently, using spaces

```
version: 1.1
source : ...
joins :
  - name: ...
    source: ...
    on: ...
dimensions :
  - name: ...
    expr: ...
measures :
  - name: ...
    expr: ...
```

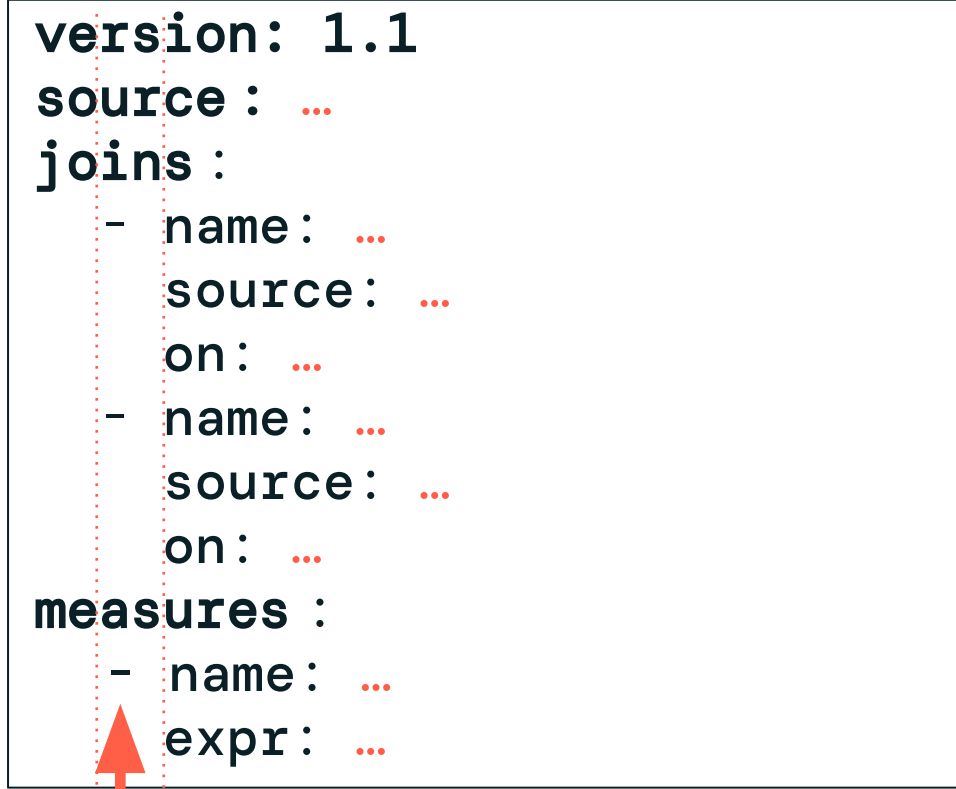


Use a consistent number
of spaces to indent.
DON'T USE TABS



To give a list item, indent and prefix with -

```
version: 1.1
source: ...
joins:
  - name: ...
    source: ...
    on: ...
  - name: ...
    source: ...
    on: ...
measures:
  - name: ...
    expr: ...
```

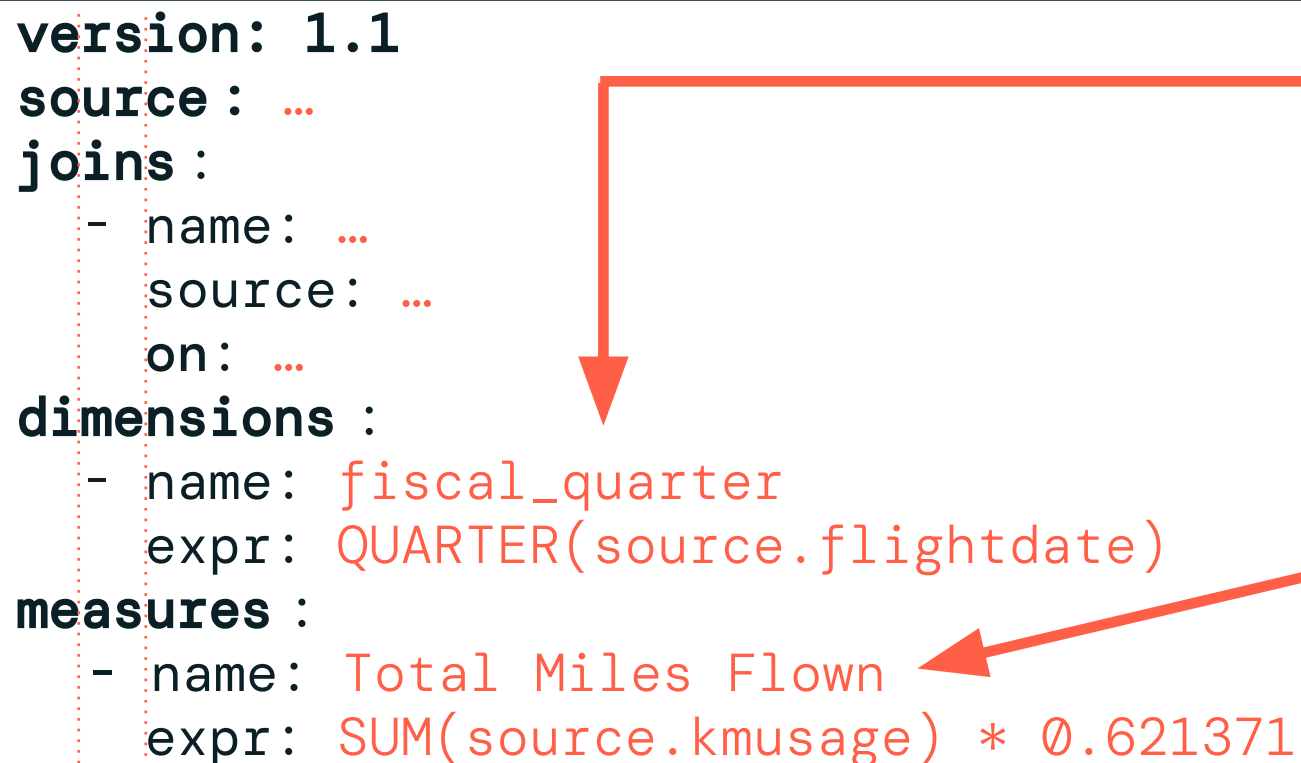


Hyphens that are indented the same number of spaces
begin and ***continue*** a list



Spaces in field values don't need quotes

```
version: 1.1
source: ...
joins:
  - name: ...
    source: ...
    on: ...
dimensions:
  - name: fiscal_quarter
    expr: QUARTER(source.flightdate)
measures:
  - name: Total Miles Flown
    expr: SUM(source.kmusage) * 0.621371
```



Choose names that are simple and clear from the perspective of consumers

Although names you define can include spaces, there's a better practice

Use spaces freely for readability in SQL expressions



Flag multiline expressions with |

```
version: 1.1
source: ...
joins:
  - name: ...
    source: ...
    on: ...
dimensions:
  - name: fiscal_quarter
    expr: QUARTER(source.flightdate)
measures:
  - name: Total Miles Flown
    expr: SUM(source.kmusage) * 0.621371
  - name: total_revenue
    expr: |
      ROUND(SUM(source.saleprice
      * (1 - source.discount)), 2)
```

This expression continues until you reduce the indentation



A metric view YAML's top-level structure

version: 1.1

The version of Databricks's metric view API
(not your version of this metric view)

source : *catalog.schema.table*



measures :

- **name :** *Human Friendly **Measure** Name*
- **expr :** *SQL aggregation on fields from source*
- *and more*

The keyword **source** points
to the fact
table defined
above



dimensions :

- **name :** *Human Friendly **Dimension** Name*
- **expr :** *SQL expression on fields from source*
- *and more*



How that might look in the real world

version: 1.1

source: airoperations.equipmentstats.equipmentusage



measures:

- **name:** total_miles_flown
expr: SUM(source.kmusage) * 0.621371
- *and more*

Enforces uniformity in
converting from km to
miles



dimensions:

- **name:** operating_quarter
expr: QUARTER(source.flightdate)
- *and more*

Consumers of this metric
view don't have to do
date arithmetic



What if we need to enrich the fact table?

version: 1.1

source : *catalog.schema.table*

joins :

- **name :** *lookuptable*

- source :** *catalog2.schema2.table2*

- on:** *lookuptable.primarykey* = source.*foreignkey*

measures :

- ...

dimensions :

- ...

By definition, any join listed here
is a LEFT JOIN

Needn't be in the same catalog or
schema, or even the same storage,
but must be in Unity Catalog



What if we need to enrich the fact table?

version: 1.1

source : *catalog.schema.table*

joins :

- **name :** *lookuptable*
source : *catalog2.schema2.table2*
on: *lookuptable.primarykey* = source.*foreignkey*



measures :

- **name :** *measure_name*
expr: *SQL aggregation on fields from* source **or joins**



dimensions :

- **name :** *dimension_name*
expr: *SQL expression on fields from* source **or joins**



A realistic example of a join

version: 1.1

source: airoperations.equipmentstats.equipmentusage

joins:

- **name:** operators
 - source:** airoperations.equipmentregistry.aircraft
 - on:** operators.l_serialnumber = source.a_serialnumber

measures:

- ...



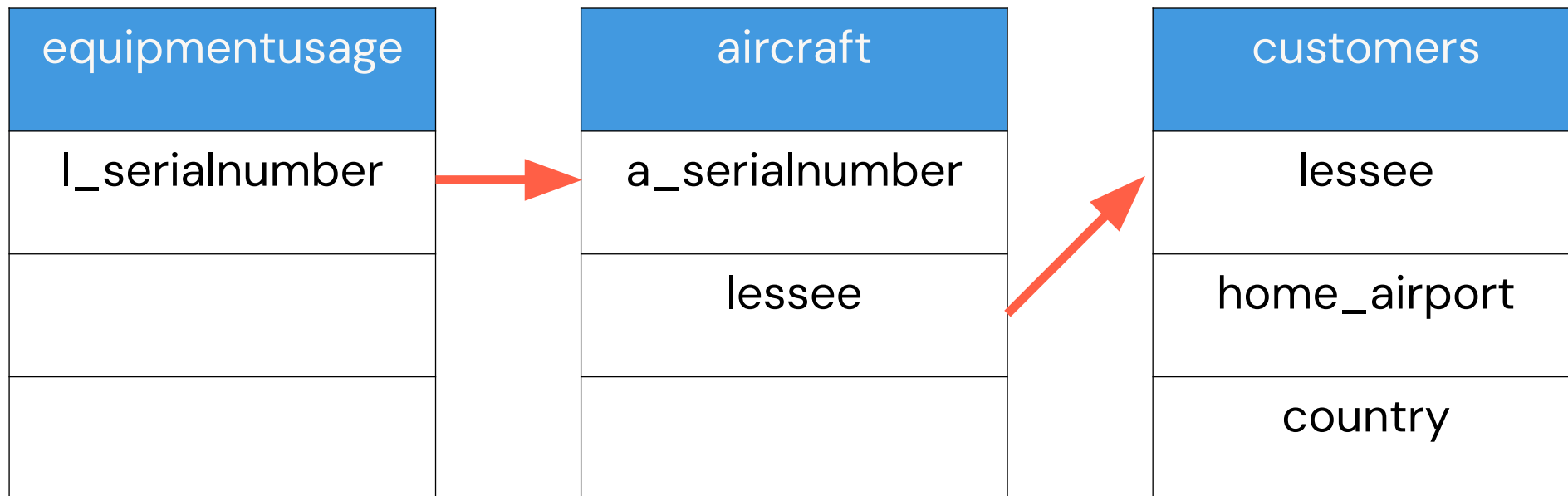
dimensions:

- **name:** operating_airline
 - expr:** operators.lessee

Now consumers of this metric view
can drill down by the airline,
for example in pie charts



What if I have a snowflake schema?



We can nest joins

version: 1.1

source : *catalog.schema.table*

joins :

- name : *lookuptable*

source : ...

on: *lookuptable.pk* = source.*fk*

joins :

- name : lookup2table_name

source : ...

on: *lookup2table.pk* = *lookuptable.fk*



A realistic example of a nested join

version: 1.1

source: airoperations.equipmentstats.equipmentusage

joins:

- **name:** operators

- source:** airoperations.equipmentregistry.aircraft

- on:** operators.l_serialnumber = source.a_serialnumber

- joins:**

- **name:** customers

- source:** finops.crm.customers

- on:** operators.lessee = customers.lessee



What if we want to filter the fact table?

version: 1.1

source : *catalog.schema.table*

joins :

- ...

filter : *SQL expression on fields from source or joins*

measures :

- ...

dimensions :

- ...



For example, let's skip old flight records

version: 1.1

source: airoperations.equipmentstats.equipmentusage

joins:

- ...

filter: YEAR(source.flightdate) >= 2025

measures:

- ...

dimensions:

- ...



...and focus on only one airline

version: 1.1

source: airoperations.equipmentstats.equipmentusage

joins:

- **name:** operators

- source:** airoperations.equipmentregistry.lessees

- on:** operators.l_serialnumber = source.e_serialnumber

filter: |  Multi-line expression
YEAR(source.flightdate) >= 2025 AND
operators.airline = "East Coast Airways"

measures:

dimensions:

- ...



You can also filter within a measure

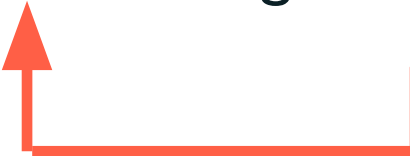
version: 1.1

source: airoperations.equipmentstats.equipmentusage

...

measures:

```
- name: miles_flown_from_heathrow
  expr: |
    SUM(source.kmusage) * 0.621371
    FILTER (WHERE source.origin = "LHR")
```



Prefix fields with **source** or
join names



How to add a comment to a YAML file

version: 1.1

Python-style comments won't be saved in UI

-- SQL-style comments are a syntax error

NO

source: airoperations.equipmentstats.equipmentusage

comment: **Instead, supply a comment field like this**

YES

dimensions :

- name: operating_airline
expr: operators.lessee

comment : **Or on individual dimensions or measures**

YES



Add enriching semantics for consumers



measures:

- **name:** total_miles_flown
- expr:** SUM(source.kmusage) * 0.621371
- display_name:** Total Miles Flown
- synonyms:**
 - distance flown in miles
 - miles traveled
- format:**
 - decimal_places:**
 - type:** exact
 - places:** 1

AI/BI will prefer this name for display

Synonyms help AI match user intents



Summary: key components of a metric view

version	The version of Databricks's metric view specification this definition uses. Helps maintain backwards compatibility.
source	The fact table against which this metric view is defined.
joins	Other tables to be LEFT JOINED to the fact table to enrich it.
filter	A SQL expression that limits the number of rows this metric view will query. Helps <i>both</i> with business focus <i>and also</i> with cost control.
dimensions	The ways in which business users will be able to slice those measures . <i>Don't include dimensions that have no business need.</i>
measures	The KPIs this metric view delivers to its business users. <i>Don't include measures with no business need.</i>



Review: guidelines for choosing measures and dimensions



Choose measures that...

- Map directly to the business's KPIs
- Need uniform definitions and computations across the business
- Are computed based solely on high-quality gold-layer data



Choose dimensions that...

Divide the measures in the ways the business cares about

- date range
- region
- product



Best practice: Avoid null values in metric views

The semantics of nulls are typically undefined, so they don't help the business

Ensure that source tables have been scrubbed of nulls

Strongly prefer gold-layer tables for building metric views

Invest in data hygiene and semantic alerts at lower layers

Don't introduce new nulls

LEFT JOINs introduce nulls for rows that don't have a match in the right-side table

SQL CASE statements emit null if they don't have a match





Defining and Consuming Metric Views

LECTURE

Create metric views



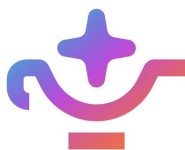
How do we create a metric view?

Overview



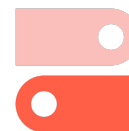
Catalog Explorer

Manually create and edit the metric view using the catalog explorer



Genie Code

Use natural language to ask genie code to create the metric view



SQL DDL

Write your own SQL query to create the metric view

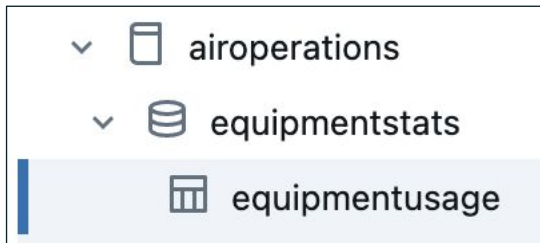




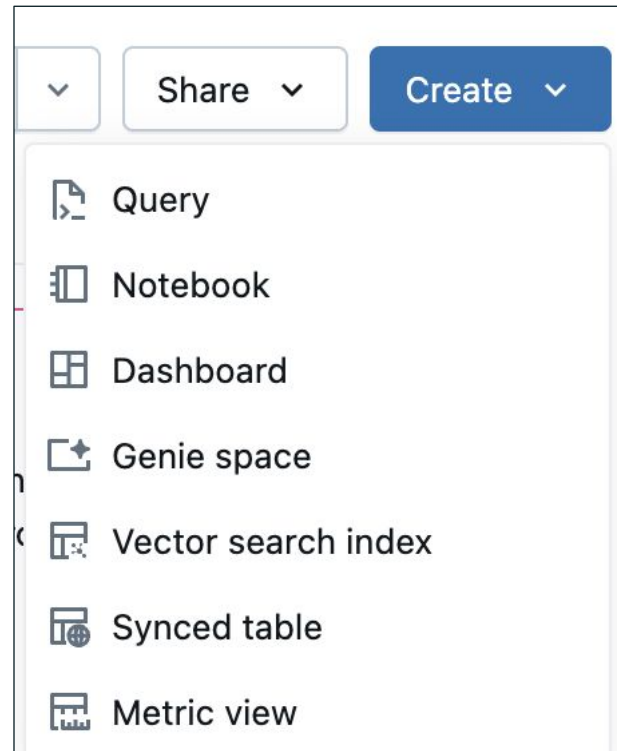
How do we create a metric view?

Method 1 – Use the Catalog Explorer

1 Navigate to fact table in Catalog



2 Choose **Create** > **Metric view**



3 Enter metric view name, catalog and schema. Click **Create**

Create metric view [X]

Name*

equipmentusage_metric_view

Location*

labuser14836665_17770316... [X] [v] default [X] [v]

Cancel Create





How do we create a metric view?

Method 2 – Use SQL DDL

- 1 Prepare your YAML in a text editor or retrieve from git

```
version: 1.1
source: airoperations.equipmentstats.equipmentusage
dimensions:
  - name: Serial Number
    expr: e_serialnumber
  - name: Flight Date
    expr: flightdate
measures:
  - name: Total Miles Usage
    expr: SUM(kmusage) * 0.621371
```

- 2 In SQL Editor, compose your DDL

```
CREATE OR REPLACE VIEW usage_metric_view (
  `Serial Number`,
  `Flight Date`,
  `Total Miles Usage`
)
WITH METRICS
LANGUAGE YAML
AS $$
YAML goes here
$$
```

List all metrics
and dimensions

- 3 Paste in YAML between the \$\$ delimiters

- 4  Run the DDL statement you composed

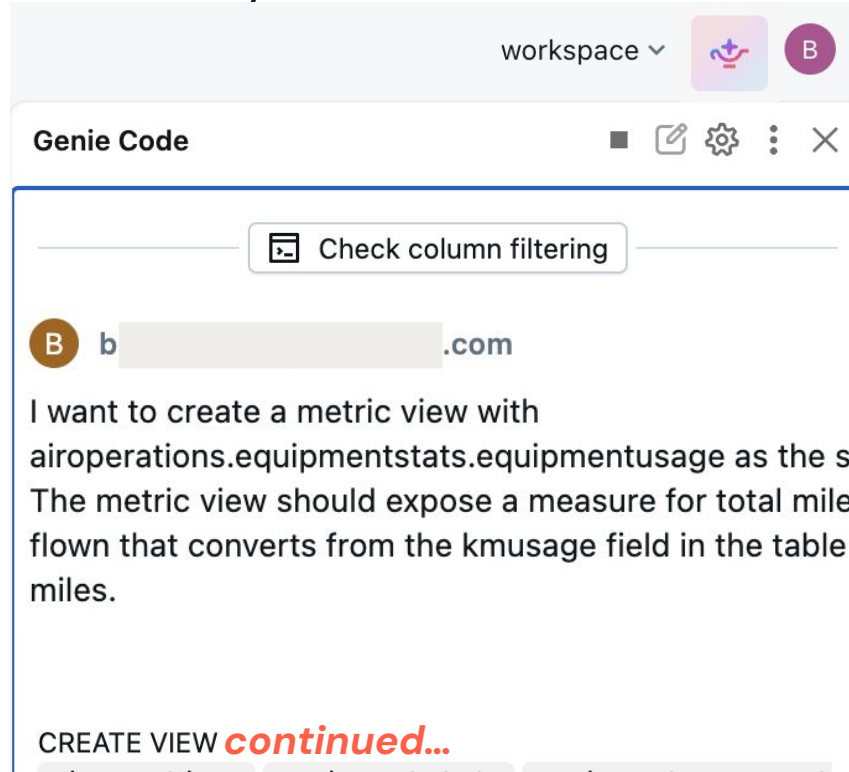




How do we create a metric view?

Method 3 – Tell the Genie Code with natural language

- 1 Click the Genie Code icon and describe your desired metric view in



- 2 Paste the result into the **Create > Metric View** dialogue.

- 3 Review and edit Genie Code's proposal.

- 4 Click **Create**



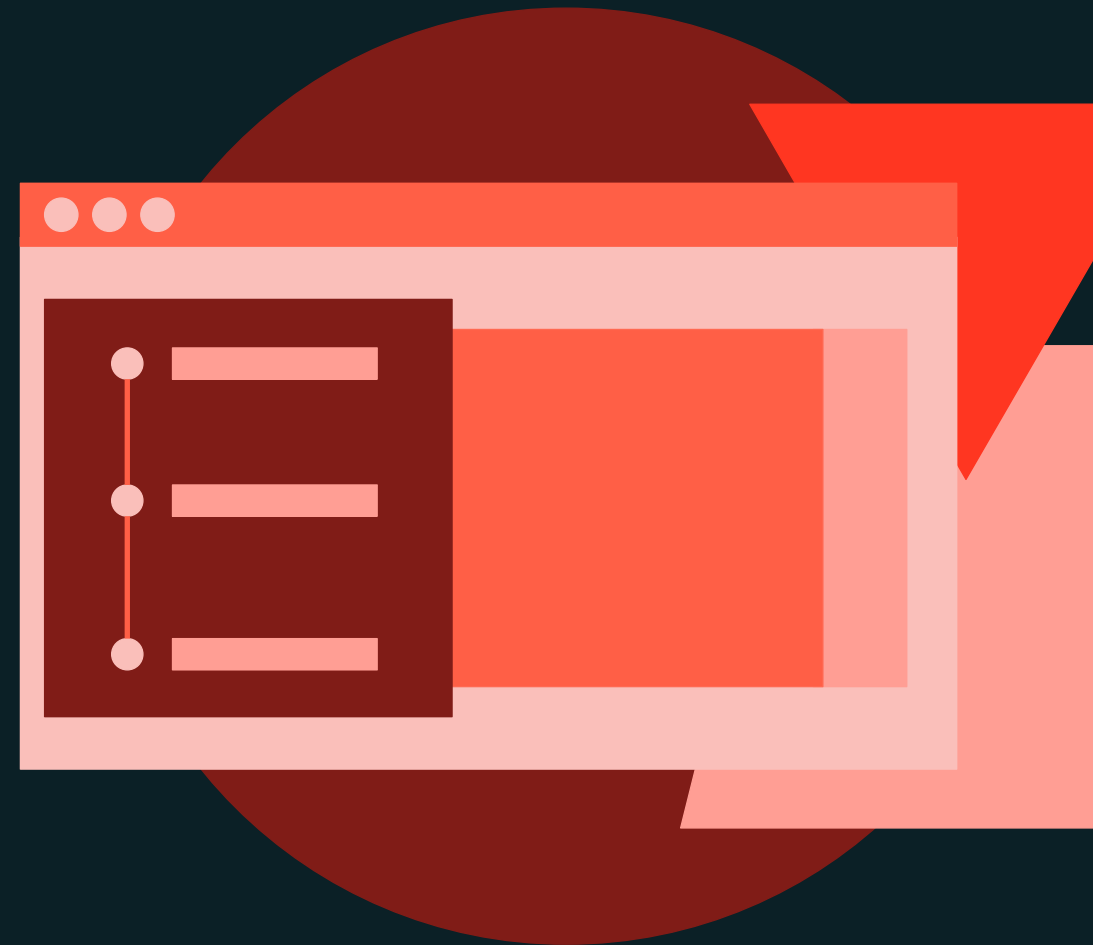


Defining and Consuming Metric Views

DEMONSTRATION

Create a simple metric view

If you have access to the Vocareum lab environment, feel free to follow-along to with this experience.



Notebook: 1 Demo – Create a Simple Metric View



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Defining and Consuming Metric Views

LECTURE

Query and consume metric views



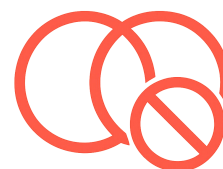
Know these limitations of metric views

Metric views are *data products*, so don't apply data-engineering concepts



SELECT * not permitted on metric views

- **Wildcards** in queries apply to table fields, not to business metrics.
- To **see which fields** are part of a metric view,
 - Select **Sample data** or
 - **Open the metric view definition.**



Query-time JOINS on metric views

- Metric views cannot be directly joined with other tables at query time.
- If a **needed join is missing**:
 - Update the metric view definition or
 - Create a new metric view.

Workaround: Wrap the metric view query with GROUP BY in a CTE, then JOIN the CTE result.



Query your metric view in SQL Editor

Metric views are queried just like ordinary views

Use Catalog Explorer to show names of measures and dimensions

equipmentusage_metric_view			
Overview	Details	Permissions	Lineage
Description			
Add description			
Measures (2)			
Name	Type	Com...	Tags
Total Miles Usage	decimal(27,6)		
Average Miles Usage per Flight	double		

Use `backticks` if measure and dimension names contain spaces

```
SELECT
    `Flight Date`,
    MEASURE(`Average Miles Usage per Flight`)
FROM equipmentusage_metric_view
GROUP BY `Flight Date`
ORDER BY `Flight Date`;
```



Create a visualization in the SQL editor

Create a graph of a metric view's results then save it to a dashboard

In SQL Editor, click + above a query result and select Visualization

Raw results ▾ +

	1 ² ₃ dropout		
1			
2			
3			
4		16	11
5		0	55

- Visualization
- Filter
- Parameter

- ✓ Set type
- ✓ Choose measures
- ✓ Aggregate using dimensions
- ✓ Edit colors, titles

Click the down arrow and select Add to dashboard



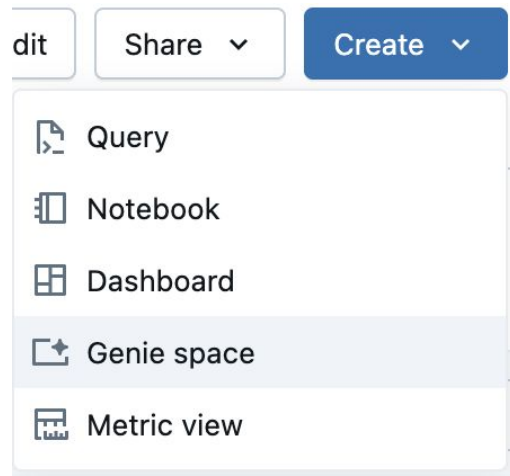
Interact with your metric view using Genie

An ad-hoc Genie space allows to you explore using natural language

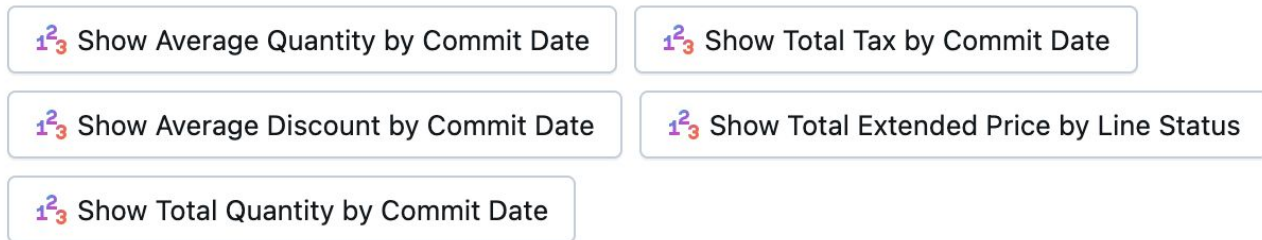
In Catalog Explorer view metric view and click Ask Genie



Or click down arrow in Create button and choose Genie space



Genie uses metric view to propose starter questions for chat



Always review the accuracy of responses.



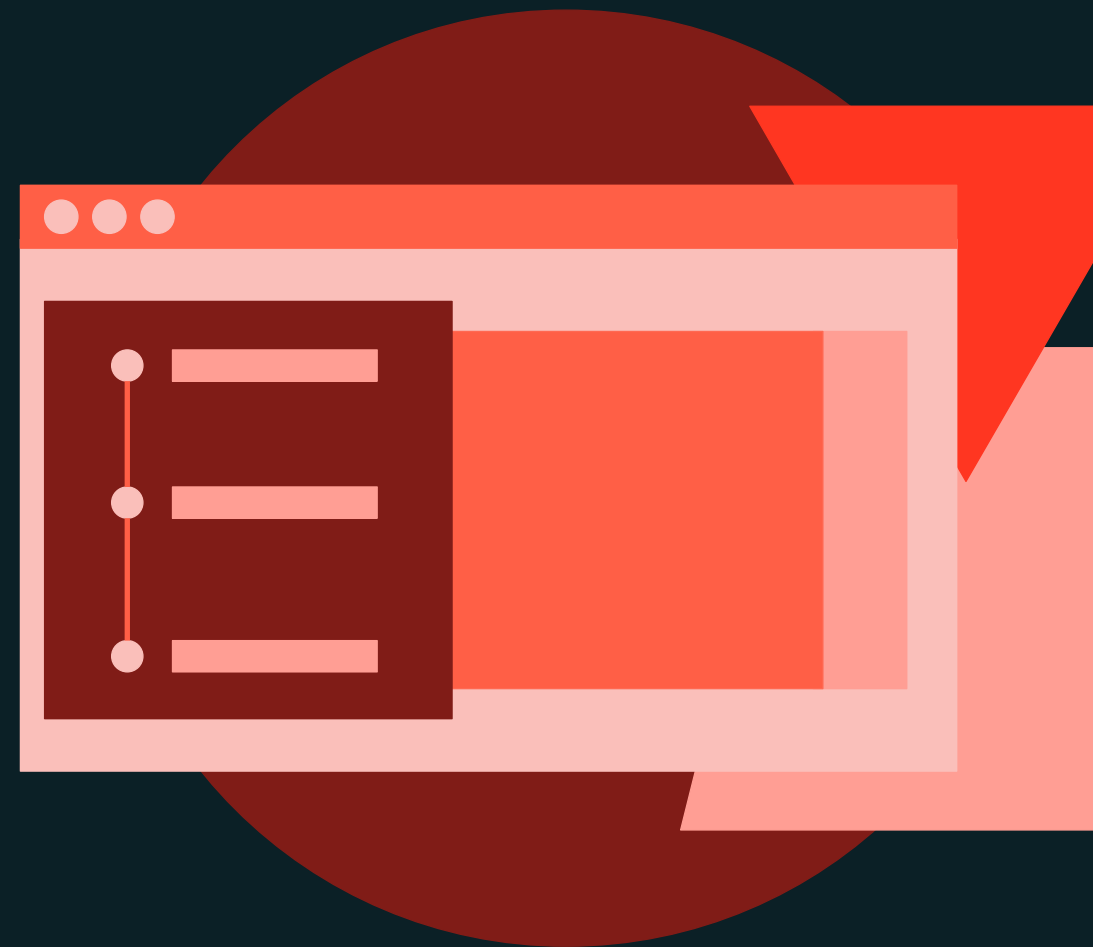


Defining and Consuming Metric Views

DEMONSTRATION

Consume a Simple metric view

If you have access to the Vocareum lab environment, feel free to follow-along to with this experience.



Notebook: 2 Demo – Consume a Simple Metric View



© Databricks 2026. All rights reserved. Apache, Apache Spark, Spark, the Spark Logo, Apache Iceberg, Iceberg, and the Apache Iceberg logo are trademarks of the [Apache Software Foundation](#).



Defining and Consuming Metric Views

LAB EXERCISE

Create and consume a Simple metric views

In this lab activity you will create a metric view and use it in various contexts.

- Open up Vocareum through Databricks Academy.
- Open the required notebook and follow the instructions in the notebook.

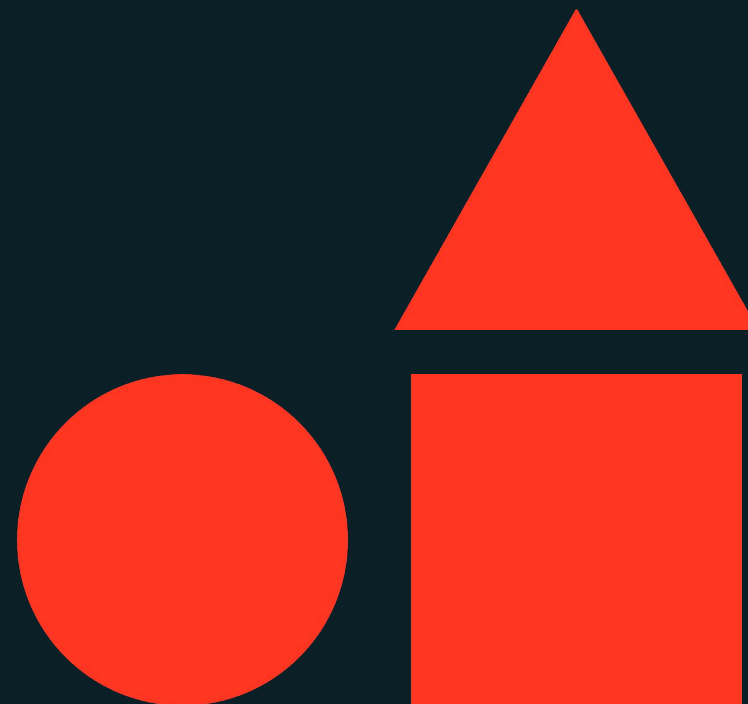
Notebook: 3 Lab – Create and Consume a Simple Metric View





Designing and Composing Metric Views

Building Semantic Models in Databricks with UC Metric Views



Section Learning Objectives

- Identify applications for metric views
- Build metric views that support business requirements
- Use composability to make metric views more maintainable



Agenda

Module Name	Lecture	Demo	Lab
Composability features of metric views	✓		
Define and Use Computed and Composed Measures		✓	
Using window measures to meet business needs	✓		
Define and Use Windowed Measures		✓	
Define Composed and Windowed Measures			✓





Designing and Composing Metric Views

LECTURE

Composability features of metric views



Use definitions of dimensions in new ones

Avoids error; fosters consistency; makes logic clearer

dimensions:

- name: order_month
 expr: MONTH(source.orderdate)
- name: fiscal_quarter
 expr: |

CASE

WHEN order_month IN (11,12,1) THEN "4"

WHEN order_month IN (2,3,4) THEN "1"

WHEN order_month IN (5,6,7) THEN "2"

WHEN order_month IN (8,9,10) THEN "3"

ELSE "Data Integrity Error"

END

Order Month
defined here...

and used here

We should never
see this,
so make it obvious



Use definitions of measures in new ones

Avoids error; fosters consistency; makes logic clearer

measures:

- name: total_revenue
expr: SUM(source.ordertotal)
- name: total_tax
expr: SUM(source.tax)
- name: total_aftertax_revenue
expr: MEASURE(total_revenue) - MEASURE(total_tax)

Measures
defined here...

...and used here
Note required syntax for
references to measures



Entire metric views can be based on others

Allows different teams to own the definitions of dimensions and measures

source: salesadmin.sales_territories.us_territories

Source is another metric view owned by a different team

filter: ordercountry = "US"

dimensions:

Applies row-level filtering to the inherited metric view

- name: us_region
 expr: region
- name: sales_rep
 expr: repname

Reuses definitions from the source metric view

measures:

- name: total_revenue
 expr: SUM(revenue)

Reuses or re-exposes measures from the source metric view





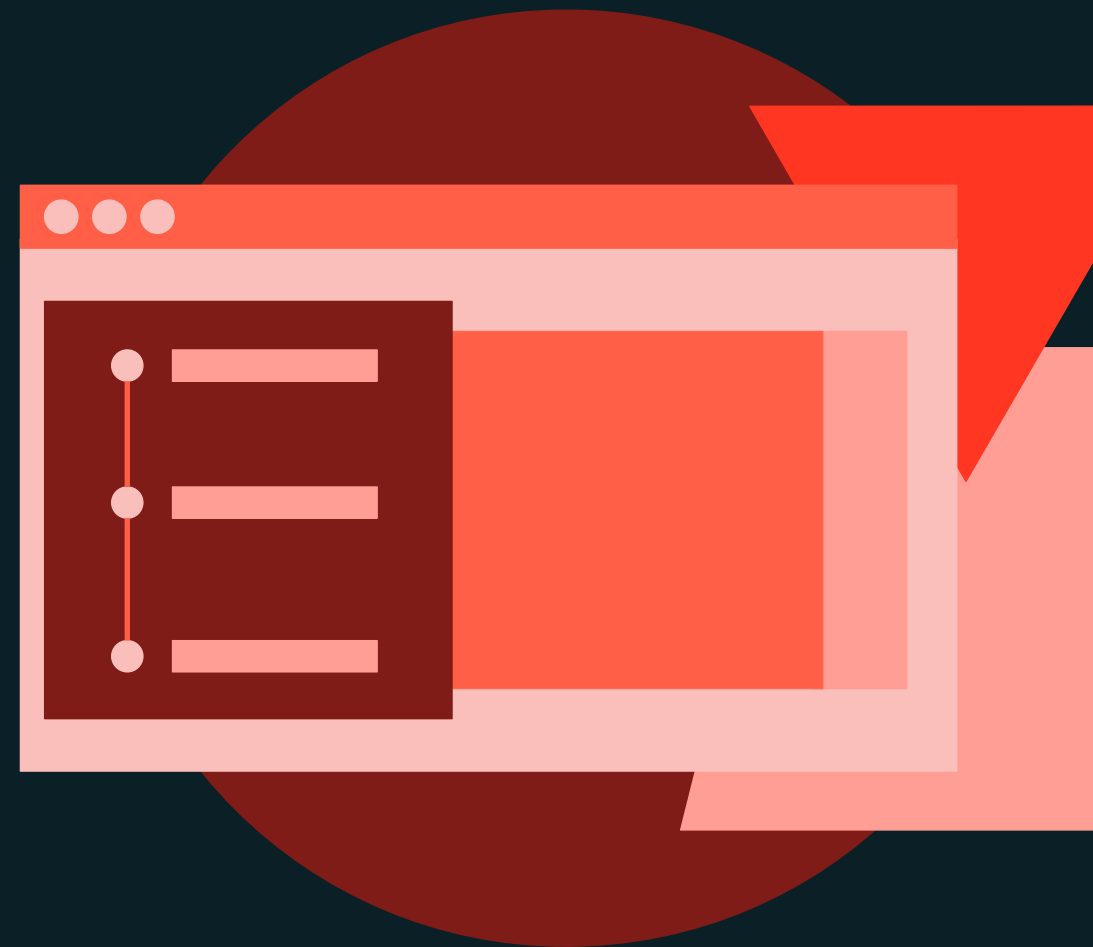
Designing and Composing Metric Views

DEMONSTRATION

Define and Use Computed and Composed Measures

If you have access to the Vocareum lab environment, feel free to follow along with this experience.

Notebook: 4 Demo – Define and Use Computed and Composed Measures





Designing and Composing Metric Views

LECTURE

Using window measures to meet business needs



SQL window queries require error-prone complexity

```
WITH sales_by_year AS (  
  SELECT YEAR(o_orderdate) as year,  
         SUM(o_totalprice) as revenue  
  FROM samples.tpch.orders  
  GROUP BY year  
) SELECT year, revenue,  
        LAG(revenue) OVER (ORDER BY year)  
        as revenue_prev_year,  
        (revenue - LAG(revenue)) OVER (ORDER BY year)  
        as revenue_difference  
FROM sales_by_year ORDER BY 1;
```

Define **Common Table Expression** **sales_by_year**

Use **LAG()** to access each year's predecessor

Return results from the **CTE**



Metric views know this pattern and encapsulate it for you

```
version: 1.1
source: samples.tpch.lineitem
dimensions:
  - name: order_date
    expr: o_orderdate
measures:
  - name: previous_year_sales
    expr: SUM(o_totalprice)
window:
  order: order_date
  range: trailing 1 year
  semiadditive: last
```

Dimension
defined here

and used here

```
name: current_year_sales
  expr: SUM(o_totalprice)
window:
  order: order_date
  range: current
  semiadditive: last
```



Now it's easy to define a year-over-year measure

measures:

- name: yoy_sales_change
expr: MEASURE(current_year_sales) - MEASURE(previous_year_sales)
- name: yoy_sales_change_pct
expr: 100.0 * MEASURE(yoy_sales_change)/MEASURE(previous_year_sales)



It's also easy to define year-to-date measures

dimensions :

- name: order_date
expr: o_orderdate
- name: order_year
expr: DATE_TRUNC('year', o_orderdate)

measures :

- name: ytd_sales
expr: SUM(o_totalprice)

window:

- order: order_date
range: cumulative
- order: order_year
range: current

Two windows
are required

Accumulates sales
over the order date

Resets the
accumulation each
year



Understanding window-measure syntax

<i>Field</i>	<i>Values</i>	<i>Meaning</i>
order	A dimension	What axis to compare along
range	current	The current row
	trailing <i>number unit</i> or leading <i>number unit</i>	Includes rows from the current row going backward or forward by the specified <i>number</i> of time <i>units</i> , NOT including the current row
	cumulative	All rows with order values less than or equal to the current value
	all	All rows
semiadditive	first or last	Which value to choose from a range when we aren't aggregating by the order column



What does semi-additive mean?

- What should happen when the metric view is being queried with a GROUP BY clause that doesn't include your window function's **order**?
- Decide which is most useful or least surprising, **semiadditive: first** or **semiadditive: last**





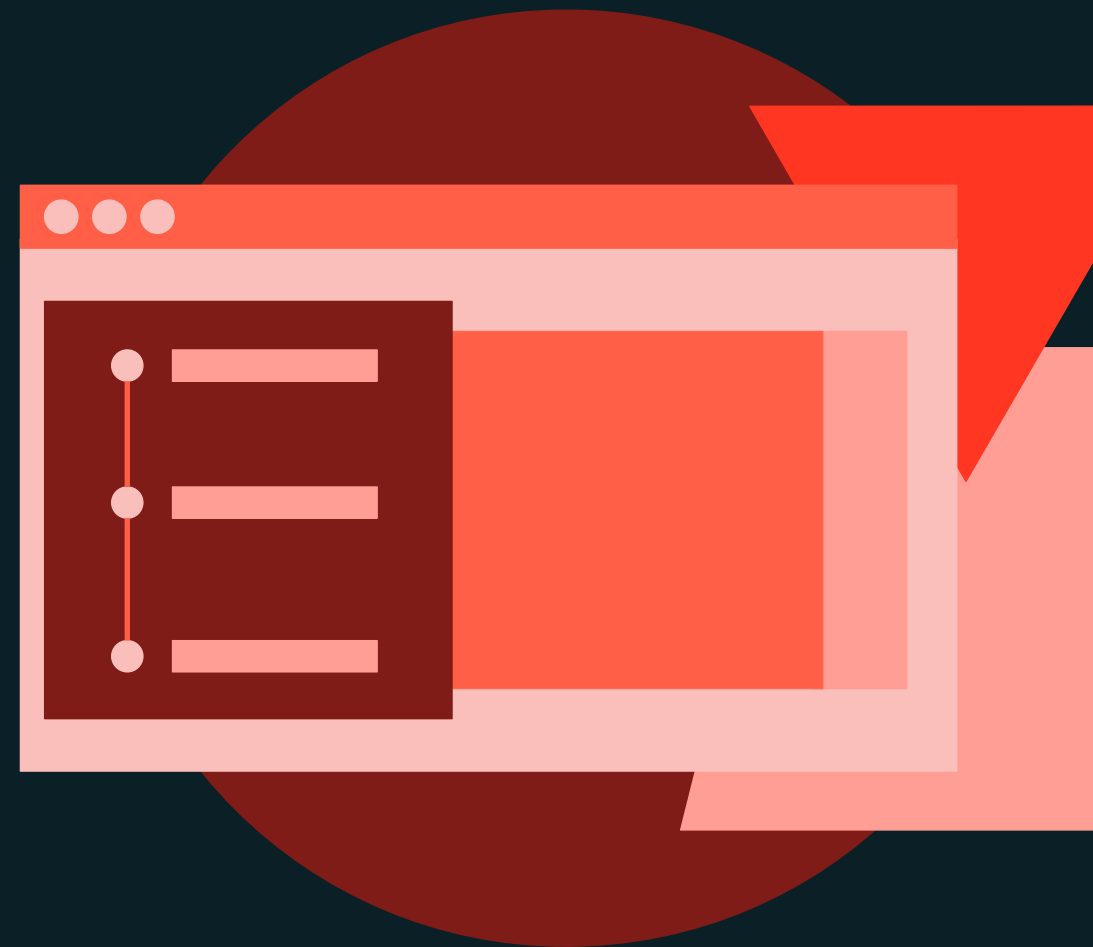
Designing and Composing Metric Views

DEMONSTRATION

Define and use windowed measures

If you have access to the Vocareum lab environment, feel free to follow along with this experience.

Notebook: 5 Demo – Define and Use Windowed Measures





Designing and Composing Metric Views

LAB EXERCISE

Define Composed and Windowed Measures

In this lab, you will [one sentence quick summary]

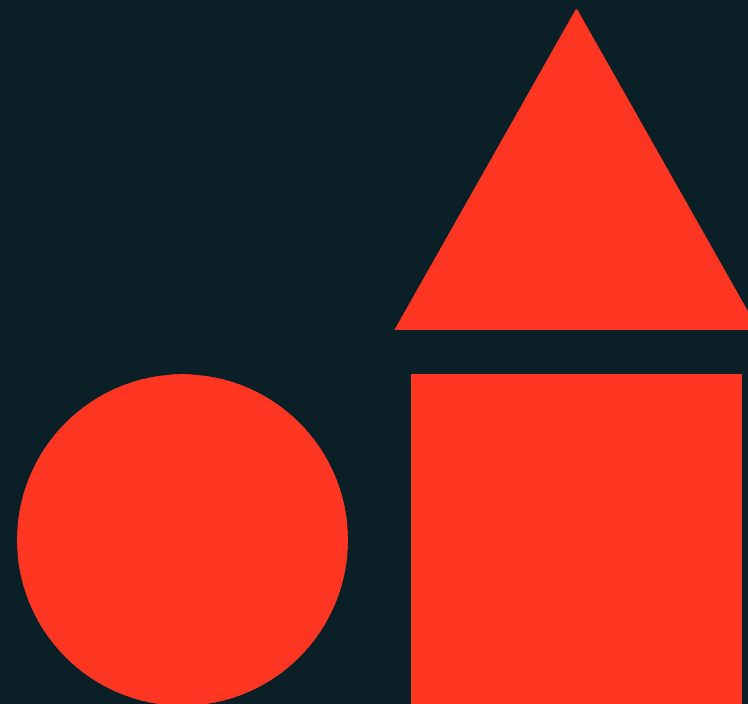
- Open up Vocareum through Databricks Academy.
- Open the required notebook and follow the instructions in the notebook.

Notebook: 6 Lab – Define Composed and Windowed Measures



Metric Views in Production

Building Semantic Models in Databricks with UC Metric Views



Section Learning Objectives

- Manage a metric view throughout its lifecycle
- Administer permissions on a metric view, both the metric view itself and its contents
- Validate a metric view and avoid common issues
- Certify a metric view



Agenda

Metric Views in Production	Lecture	Demo	Lab
Testing, validating, and certifying metric views	✓		
Managing the performance of metric views	✓		
Bringing metric views to data consumers	✓		
Additional Features Overview	✓		





Metric Views in Production

LECTURE

Testing, validating, and certifying metric views



The lifecycle of a metric view

Assess

Identify a need of the business to address with a metric view

Define

Stakeholders work with data and AI team to agree on

- what KPIs to expose
- how to define them
- how to aggregate them

Test

After tests and signoff, mark the metric view as certified

Optimize

Measure the performance and cost of the metric view in use
if necessary add filters or change the aggregations offered

Monitor

Set up sentinel alerts on key semantics
Track usage, retire when no longer useful



Test and validate metric views

Unit tests

- ✓ Null-free results
- ✓ All values returned are semantically meaningful
- ✓ Plausible results from querying each measure against each dimension

Integration tests

- ✓ Check KPIs against known correct values
- ✓ Ensure no surprising changes in published dashboards

Acceptance tests

- ✓ Designate a representative of the business to confirm that the metric view gives expected and plausible results on queries important to the business





Metric Views in Production

LECTURE

Managing the performance of metric views



Performance starts with metric views' use

Check query history for queries and pipelines that use your view

Query History

User

Last 30 days

Compute: Pipeline compute

Duration

Status

Statement

Statement ID

65 queries

Query	Started at	Duration	Source	Completed
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/28/2025, 07:4...	1.08 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
REFRESH STREAMING TABLE main.dbdemos_dlt_cdc.scd2_customers /* FLOW Expand query text	8/28/2025, 07:4...	1.05 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/28/2025, 07:4...	1.31 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/28/2025, 07:4...	4.20 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/27/2025, 07:4...	1.16 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/27/2025, 07:4...	1.08 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/27/2025, 07:4...	1.40 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline
> REFRESH STREAMING TABLE main.dbdemos_dlt_cdc...	8/27/2025, 07:4...	4.49 s	> revenue job / ... / dbdemos_dl...os_dlt_cdc	Pipeline

Click on a pipeline or query to drill in



Analyze execution plans with Query Profile

Query History

User: Me (brian.rice+1234@databricks.com) ✖ Last 7 days ⚡ Compute ▾

19 queries [Reset filters](#)

Query	Started at	Duration	Source
✓ > select order_date, measure(total_quant...	11/15/2025, 0...	2.91 s	New Query 20
✗ > select order_date, sum(measure(total_q...	11/15/2025, 0...	410 ms	New Query 20
✗ > select order_date, sum(measure(total_q...	11/15/2025, 0...	423 ms	New Query 20
✗ > select order_date, sum(measure(total_q...	11/15/2025, 0...	892 ms	New Query 20
✓ > --- describe table extended `dbacad...	11/15/2025, 0...	442 ms	Databricks Catalog
✓ > --- CREATE VIEW `yielding_pat`.`defa...	11/15/2025, 0...	2.24 s	Databricks Catalog
✓ > --- describe table extended `dbacad...	11/15/2025, 0...	570 ms	Databricks Catalog
✓ > --- describe detail `dbacademy_austr...	11/15/2025, 0...	635 ms	Databricks Catalog
✓ > --- describe detail `dbacademy_austr...	11/15/2025, 0...	2.91 s	Databricks Catalog
✗ > select `Supplier Key`, sum(MEASURE(`To...	11/14/2025, 1...	357 ms	New Query 20
✗ > select `Supplier Key`, sum(`Total Quan...	11/14/2025, 1...	713 ms	New Query 20
✗ > select sum(`Total Quantity`) from (sel...	11/14/2025, 1...	356 ms	New Query 20
✗ > select sum(`Total Quantity`) from (sel...	11/14/2025, 1...	393 ms	New Query 20

ID: 01f0c2ac-2721-1d3f-a7f9-051070bef17b [📄](#)

```
select
  order_date,
  ...8 more lines
```

Bytes read 88.33 KB	Bytes written 0
Rows read 5,276	Rows written 0
Files read 1	Files written 0

[See query profile >](#)

[See longest operations for this query](#)

Query wall-clock duration ⓘ

Total wall-clock duration	2 s 906 ms
> Scheduling ⓘ	3% 81 ms
Optimizing query & pruning files ⓘ	19% 560 ms
Executing ⓘ	78% 2 s 265 ms

Start time Nov 15, 2025, 09:21:49 PM GMT-08:00
End time Nov 15, 2025, 09:21:52 PM GMT-08:00
Result fetching by client ⓘ 42 ms

[See query profile](#)



Best practices for metric view performance

Use filters to reduce number of rows scanned

- Limit the scope of the metric view to what the business needs
- If different subgroups need access to different ranges of rows, consider offering one metric view for each

Ensure that joined lookup tables are as small as possible

- If the tables that you join to your fact table are small, joins can be executed as broadcast joins
 - Much cheaper than shuffle joins
 - Broadcast joins will show up in query profile



Materialize metric views

Materialization is available as “Experimental Release”.

Materialized views offer an important tradeoff:

Compute	Reduced usage, with increased control
Query latency	Decreased
Storage	Increased usage
Cost control	Improved
Freshness of data	Under your control

Because metric views are intended to focus on KPIs important to the business, these tradeoffs will be especially important for them





Metric Views in Production

LECTURE

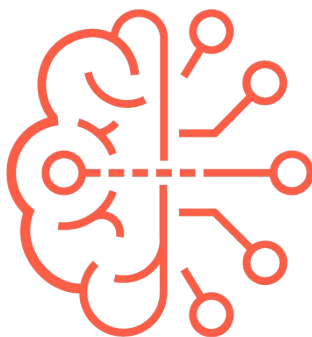
Bringing metric views to data consumers



How do users consume a metric view?



***Using an
AI/BI dashboard***



***As the basis of a
Genie space***



***As the driver of an
alert***

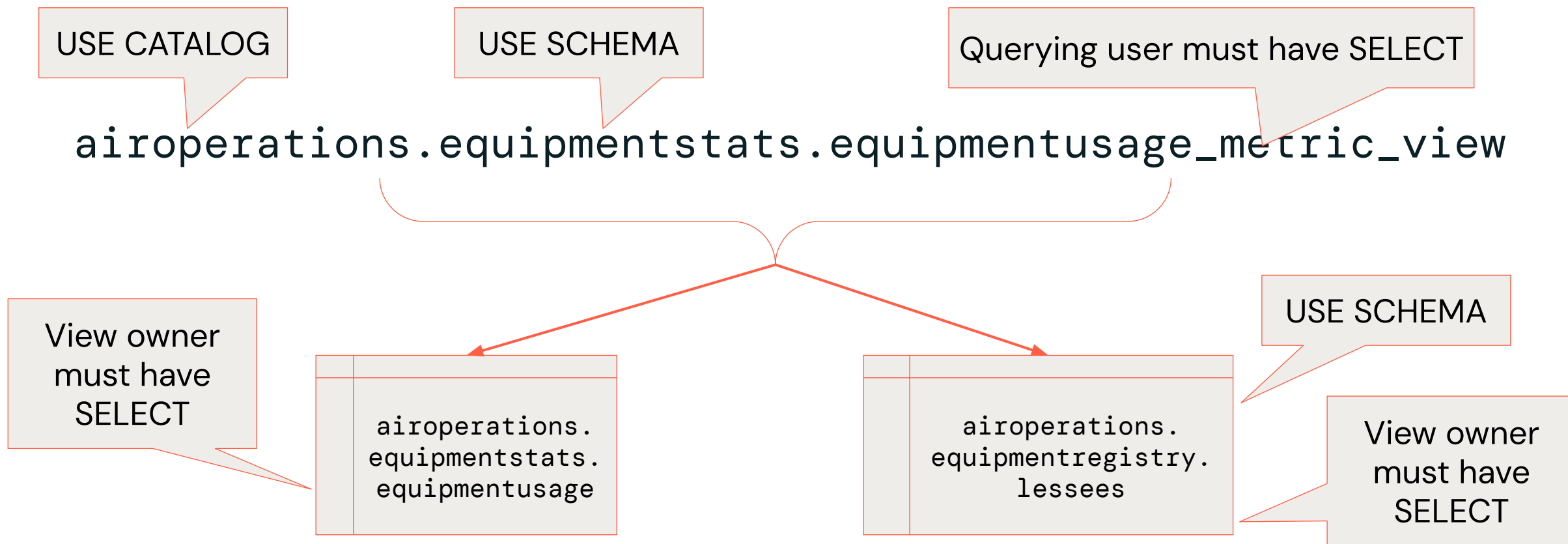


***Through a
saved query***



What permissions do users need?

Manage using Permissions tab in Catalog Explorer, or else SQL Editor



How do we create a dashboard?

Use the Databricks UI to visualize measures from the metric view

The screenshot shows the Databricks 'New Dashboard' interface. At the top, the title is 'New Dashboard 2026-04-24 19:51:18' with a star icon. To the right are buttons for 'Refresh', 'shared_warehouse', 'Publish', and 'Share'. Below the title bar, there's a tab labeled 'Data' with a table icon, and another tab labeled 'Untitled page' with a funnel icon. A red arrow points to the 'Data' tab with the instruction: '1. Click "Data" tab to get going'. Below this, text says: 'Create a dataset for your dashboard by writing your own SQL query or selecting an existing table.' In the center of the dashboard area, another instruction says: '2. Add a visualization or filter to your page'. Below this, text says: 'Select a tool then drag and draw to create your first dashboard widget.' A red arrow points to a toolbar at the bottom of the dashboard area. The toolbar contains icons for 'Add a visualization' (a bar chart), 'Add a text box' (a text box), 'Add a filter' (a funnel), 'Undo' (a left arrow), and 'Redo' (a right arrow). A 'Move' icon (a hand) is also present. At the bottom right of the dashboard area, there is a 'Settings' button with a gear icon.



How do we create a Genie space?

Before you start, make sure your semantics are clean

New Space ☆

Change to a useful title



New Space

[Click here to add a description](#)

📎 Attach a file (csv, xlsx)

🔗 Explain the data set

1² 3 Show average_order_amount by sales_rep

1² 3 Show average_order_amount by product_id

1² 3 Show total_quantity by customer_id

1² 3 Show total_quantity by order_date

1² 3 Show total_quantity by product_id

+ Add a sample question

These sample questions come directly from your measures and dimensions

Add more sample questions if helpful

📎 Ask your question...



Always review the accuracy of responses.



How do we create an alert?

Before you start, make sure you have business actions in mind

1. Write and run a query
Query data to view results.

2. Configure the alert
After you run the query, define the condition based on the results and set up notifications.

3. Schedule the alert
Add a schedule so that the alert is evaluated periodically.

4. Notifications
Set up notifications for the alert.



How do we create a saved query?

Before you start, have an audience in mind

The screenshot shows the Databricks SQL interface with three red callout boxes and arrows indicating the steps to create a saved query:

- Compose query pointing at metric view**: An arrow points from this box to the SQL editor area, which contains the text "1 | Start typing or generate with AI (% + I)...".
- Choose File > Share**: An arrow points from this box to the "Generate" button in the top toolbar.
- Save**: An arrow points from this box to the "Save" icon (a document with a checkmark) in the top toolbar.

Additionally, a red box labeled **Replace with descriptive name** points to the query tab title "New Query 2025-11-18 7:09pm".



Best practices: consumer-friendly metric views

Goal	Best practice
Make dashboards and Genie spaces maximally helpful to data consumers	✓ Build the metric views with a focus on expressed business problems
	✓ Be sure your data is null-free
Let AI help the consumers even more	✓ Ensure that source fields have fully populated Unity Catalog metadata
	✓ In the metric view definition, supply <code>display_name</code> , synonyms, and format hints





Building Semantic Models in Databricks with
Unity Catalog Metric Views

LECTURE

Additional Features Overview



Materializing metric views (experimental)

source: `airoperations.equipmentstats.equipmentusage`

...

materialization:

schedule: every 6 hours

mode: relaxed

materialized_views:

- **name:** `airline_breakdown`

- type:** aggregated



- dimensions:**

- `airline`



- measures:**

- `total_miles_flown`



Redacting and hiding information

Metric views respect row masking and column-level filtering

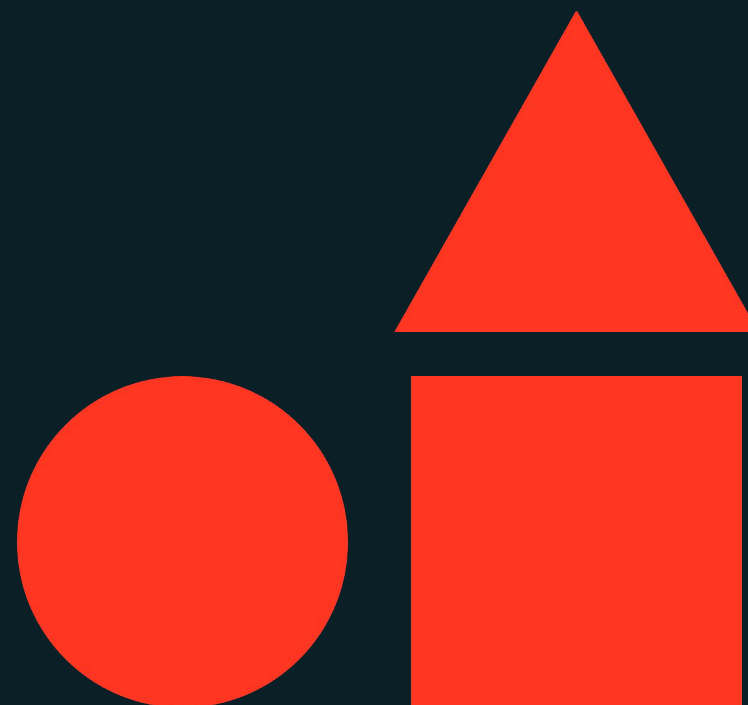
			
Name	ID	Amt paid	more...
Ted	221-04-08	41.54	
Emma	904-66-74	84.98	
 Pat	432-20-21	8.60	
Maya	811-64-51	55.00	
 Jun	656-13-07	50.09	
more...			

Metric views respect Attribute-Based Access Control (ABAC)

redact_ids			
Name	ID	Amt paid	more...
Ted	221-04-08	41.54	
Emma	904-66-74	84.98	
Pat	432-20-21	8.60	hide_EU_cus
Maya	811-64-51	55.00	
Jun	656-13-07	50.09	hide_EU_cus
more...			

Summary and Next Steps

Building Semantic Models in Databricks with UC Metric Views



Module Learning Objectives

- Review what you have learned
- Work with your new skills in a summative lab
- Understand where to go next



Agenda

Summary and Next Steps	Lecture	Demo	Lab
Review	✓		
Summative lab			✓
Knowledge check			
Next steps	✓		





Summary and Next Steps

LECTURE

Review



Course Learning Objective Recap

- Explain the role and benefits of semantic modeling with metric views in Databricks
- Define and manage metric views
- Model advanced business metrics, including windowed measures
- Integrate metric views into Databricks dashboards, Genie, and SQL workflows
- Be aware of best practices for security, governance, and maintenance of metric views



Review: why metric views?

Metric views *abstract, unify, and enforce*



- Metric views provide a smooth surface for **business data consumers**, uncluttered by engineering concerns
- Metric views are an audit-friendly environment for a single definition of all KPIs
- Metric views comprise a data product, built and maintained by the data + AI team, for the business
- Metric views offer a ***semantic*** model of the data, customized for what business decision makers ***care most about***



Review: key components of a metric view

version	The version of Databricks's metric view specification this definition uses. Helps maintain backwards compatibility.
source	The fact table against which this metric view is defined.
joins	Other tables to be LEFT JOINED to the fact table to enrich it.
filter	A SQL expression that limits the number of rows this metric view will query. Helps <i>both</i> with business focus <i>and also</i> with cost control.
measures	The KPIs this metric view delivers to its business users. <i>Don't include measures with no business need.</i>
dimensions	The ways in which business users will be able to slice those measures . <i>Don't include dimensions that have no business need.</i>





Summary and Next Steps

LAB EXERCISE

Summative lab

In this lab, you will [one sentence quick summary]

- Open up Vocareum through Databricks Academy.
- Open the required notebook and follow the instructions in the notebook.

Notebook: 7 – Summative Lab





Summary and Next Steps

LECTURE

Next steps



Next Steps

Data Analysis with Databricks

- Continue your learning through [self-paced](#) or [instructor-led](#) offerings
- The courses offer hands-on instruction in:
 - Databricks SQL
 - Lakehouse Architecture
 - Delta Lake
 - Data Security
 - Data visualizations and dashboards

Data Analysis Associate Certification

- Validate your data and AI skills on Databricks by earning a Databricks credential
- [Exam information](#) and [exam guide](#)

Databricks Performance Optimization

- Continue your learning through [self-paced](#) offerings
- The course offers hands-on instruction in assessing performance and debugging applications



Questions?



NEW

Context Engineer associate certification

Industry's first context engineering certification

Beta exam details

Validates ability to design, assemble, and govern the information AI agent systems receive at inference time

Covers the full context stack — system prompts, Vector Search retrieval, memory architectures, MCP tool integration, and context window management

Governance-forward — includes Unity Catalog, data quality, PII handling, policy enforcement, and multi-agent workflow strategies

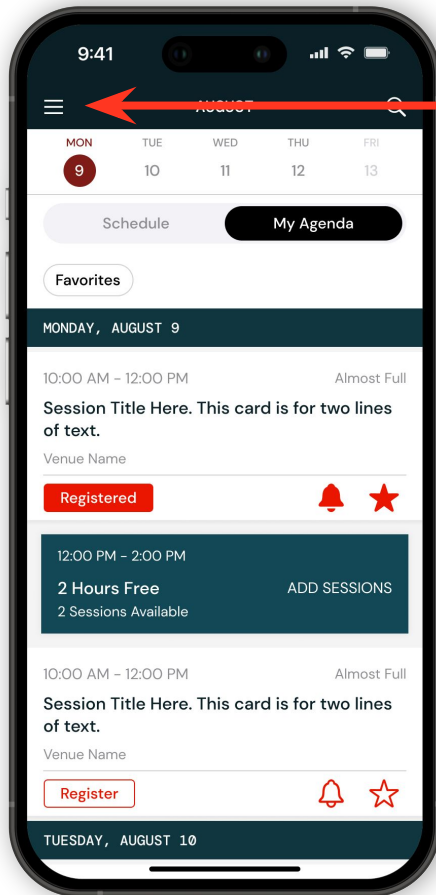


Free beta exam at Data + AI Summit (June 15–18) — one free attempt per attendee, walk-in only



Complete Your Surveys

Your feedback has a direct impact on Data + AI Summit content



- You will receive a survey for each session attended
- Open the Databricks Events app and select “My Surveys” from the menu
- Surveys can also be submitted in the Attendee Portal



